Experiment:1

Aim: Implement Bubble sort, Insertion Sort, Selection Sort

1. Bubble Sort

Definition:
Bubble Sort is a simple comparison-based sorting algorithm where each pair of adjacent elements is compared and swapped if they are in the wrong order. This process is repeated until the array is sorted. It is named "Bubble Sort" because smaller elements "bubble" to the top of the list.

Algorithm:

1. Start from the first element of the array.
2. Compare the current element with the next element.
3. If the current element is greater than the next element, swap them.
4. Move to the next element and repeat step 2 until the end of the array.
5. Repeat steps 1–4 for the remaining unsorted elements.
6. Stop when no swaps are required in a complete pass.

Python Program:

```
def bubble_sort(arr):
  n = len(arr)
  for i in range(n):
      swapped = False
      for j in range(0, n-i-1):
          if arr[j] > arr[j+1]:
              arr[j], arr[j+1] = arr[j+1], arr[j]
              swapped = True
      if not swapped:
          break
  return arr

# Example
arr = [64, 34, 25, 12, 22, 11, 90]
print("Sorted array:", bubble_sort(arr))
```

OUTPUT:
Sorted array: [11, 12, 22, 25, 34, 64, 90]

Dept of AIML

2. Insertion Sort

Definition:
Insertion Sort is a simple sorting algorithm that builds the final sorted array one element at a time. It works similarly to the way you sort playing cards in your hands.

Algorithm:

1. Start with the second element (index 1) as the key.
2. Compare the key with elements before it.
3. Move elements that are greater than the key one position ahead.
4. Insert the key into the correct position.
5. Repeat steps 1–4 for all elements in the array.

Program Code:
```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr

# Example
arr = [12, 11, 13, 5, 6]
print("Sorted array:", insertion_sort(arr))
```

OUTPUT:
Sorted array: [5, 6, 11, 12, 13]

3. Selection Sort

Definition:
Selection Sort is a sorting algorithm that divides the array into two parts: the sorted part and the unsorted part. It repeatedly selects the minimum (or maximum) element from the unsorted part and moves it to the end of the sorted part.

Algorithm:

Dept of AIML

1. Start with the first element as the minimum.
2. Search the minimum element in the unsorted part of the array.
3. Swap the found minimum element with the first element of the unsorted part.
4. Move the boundary of the sorted part by one element.
5. Repeat steps 1–4 until the entire array is sorted.

Program:

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr

# Example
arr = [64, 25, 12, 22, 11]
print("Sorted array:", selection_sort(arr))
```

OUTPUT:
Sorted array: [11, 12, 22, 25, 64]

Dept of AIML